GT School — Technical Project

Build the GT Marketing Hub on a sync engine that keeps a CRM as the source of truth

TIME BOX	FOR	DELIVERABLE	WHAT YOU GET	SERVICES
~48 hours — handed out Thursday evening, due Saturday night. The compression is deliberate.	AI-native full-stack engineers	Working system + short write-up + a recorded video walkthrough	Exactly two documents: this Brief and the Product Specification. Everything else you create yourself.	Real free-tier accounts you create yourself

At GT School we run a small software factory: a CRM holds the truth about our families, and a fleet of products read from and write back to it — dashboards, registration flows, campaigns, payment links, and analytics. The hard part is never any single feature. It is keeping many services talking end-to-end, in real time, without leaking or contaminating data between contexts that must stay separate — and then turning that trustworthy data into a product the team actually runs the business on.

This is one project in two connected phases. First you build a slice of our data backbone: a sync engine that keeps a CRM in sync with your own stores without contamination. Then you build a real product on top of it — the GT Marketing Hub — to an actual product specification our marketing lead wrote and approved. The spec ships with this brief. It is detailed on purpose: this is an execution test against a real, messy, multi-source requirement, not a blank page. Use whatever AI tools you want; we expect you to. We are evaluating the engineer driving them, not the tools.

Your source of truth: the GT Marketing Hub spec

> Attached to this brief is “GT Marketing Hub — Product Specification v2,” written by our marketing lead and approved for development. It is the requirement for Phase 2. Read it end to end before you write a line of code. It defines 13 modules, the owners of each, every data source, the user roles, the cross-module rules, and — importantly — the things that are known-broken or deferred. We are not asking you to invent the product. We are asking you to build the one that is specified, and to make the right engineering calls inside it. These two documents — this Brief and the spec — are everything you receive; together with the public web they contain all you need to complete both phases.

13 modules, real owners	Home/Command Center, Grassroots, Content, Summer Camp, Nurture, Dashboard/KPI, CRM Ops, Field Marketing, Admissions/VoC, Budget, Decision Queue, Resource Library, Website Analytics — each with a named owner and primary data source.
Three roles, hard gates	Admin (Marketing Lead), Leader (designated leadership — exclusive Decision Queue access), Operator (own module write, others read). Role gating is a requirement, not a nicety.
Many real data sources	Supabase app_form, HubSpot (CRM, Conversations, sequences, lead score, reporting), Meta Business Suite, X/Twitter, GA4, Google Sheets, summer.gt.school, community.gt.school.
Composable, per-user Home	A 30+ widget library; each user builds and saves their own dashboard from it. Default starter pack pre-checked for new users.

You cannot finish all 13 modules in ~48 hours, and we do not expect you to — the time box is deliberately tight. How you sequence the spec — which modules you build deep, which you stub, and why — is itself a judgment we are scoring. Build a coherent, working slice that honors the spec's rules, not a broad shell that violates them.

Phase 1 — The data backbone (sync engine)

Everything in the Hub stands on data you can trust, so you build that foundation first. The spec is explicit that Supabase app_form — not HubSpot field values — is the source of truth for funnel, TEFA, income, and grade, and that several HubSpot fields are unreliable. Phase 1 is where you earn the right to make that claim.

The challenge

> Build a sync engine and a thin app around it. A CRM (HubSpot) is the source of truth for "contacts" (families) and "deals" (enrollments). Your system keeps the CRM in sync, bidirectionally, with your own application database — and routes records into separate, isolated data contexts that must never bleed into one another. A payment service (Stripe, test mode) drives state changes that must propagate correctly through the whole chain.

The scenario

GT runs multiple programs at once — think Summer Camp (its own registrations and P&L) and the main Fall enrollment push. A family can be in one, the other, or both. Each program has its own data store and its own rules. The CRM is the single place where staff see everyone, but no program may ever read or write another program's private data. When a family pays, that event has to flow from the payment service, update the CRM, land in the correct program's database, and be reflected in your app — with no double-writes, no lost updates, and no cross-program contamination. The spec also requires reconciling dual sources (summer.gt.school + a registration form; HubSpot + community.gt.school) without double-counting — the same discipline.

What the backbone must do

- Sync data both directions between the CRM and your app database, and keep them consistent over time — not just on first import.
- Keep each program's data strictly isolated. A bug, a query, or a leaked key in one program must not expose or corrupt another program's data. Prove this.
- Process a payment event end-to-end: payment → CRM update → correct program store → visible in your app. Make it idempotent.
- Expose a sync-parity / data-confidence signal: the spec's CRM Ops module needs to know when Supabase and HubSpot disagree, and flag it. Build the plumbing that makes that honest.
- Handle the messy parts: retries, partial failures, duplicate webhooks, rate limits, conflicting edits, and dual-source reconciliation. Decide what "correct" means and enforce it.

Phase 2 — The product (GT Marketing Hub, to spec)

Now build the GT Marketing Hub described in the attached spec, on top of the backbone you just made. The same CRM and the same isolation and reconciliation discipline carry forward — you should not be re-solving sync in Phase 2; you should be spending it. Implement to the spec: its modules, its roles, its data rules. Where the spec leaves an engineering choice open (stack within reason, data model, what to build first), that choice is yours to make and defend.

The product, in the spec author's words

> "The GT Marketing Hub is a centralized web application where every GT Anywhere marketing function — grassroots, content, nurture, operations, events, admissions, summer camp — has its own module with task tracking, measurable outcomes, and dashboards. Leadership can monitor all functions, provide input (approve decisions, set goals, leave comments), and customize their personal view." Build that.

Implement the Hub to spec: per-function modules with their real data sources, a composable per-user Home built from the widget library, the canonical weekly KPI scorecard, the Budget Tracker where every workstream's spend reconciles to the $365K total, and the leadership-only Decision Queue. Honor the cross-module rules — single source of truth per number, auto-created cross-links, and the data-confidence banner when sync parity drops. Enrich decisions with external public-school data via Open Data (tryopendata.ai). The hard part is making 13 modules, many data sources, and many views cohere into one trustworthy place where a budget number means the same thing everywhere.

Non-negotiables from the spec

- Auth + three roles enforced: Admin, Leader, Operator. The Decision Queue module must be gated to Leaders only — Operators can submit but never view or act on the full queue.
- Single source of truth, honored: every number traces to exactly one authoritative source (e.g. funnel/income from Supabase app_form, engagement from HubSpot, budget from the Hub itself). Don't compute the same figure two ways.
- Budget Tracker reconciles: workstream rows (recommended / planned / committed / actual / remaining) sum to the total everywhere they appear; >10% variance auto-flags to the Decision Queue.
- Composable per-user Home: a widget library with a default starter pack, add/remove + saved layout per user, drawing live from the other modules.
- Real integrations, reconciled: at least HubSpot live, plus the dual-source reconciliation the spec calls for (e.g. summer.gt.school + form, or HubSpot + community). Plus a real Open Data query that changes a decision.
- Respect the known gaps: UTM attribution is broken, event-to-consult is uninstrumented, some fields are unreliable. Surface these honestly (the CRM Ops module exists for exactly this) rather than faking green.

Services to wire together (minimum)

Create your own free-tier accounts for each. Do not ask us for credentials — provisioning and securing them is part of the test. Stand in for sources you can't access (Meta, GA4, summer.gt.school) with realistic seeded data behind the same interface, and say so.

HubSpot (free developer / CRM sandbox)	Source of truth for contacts, deals, engagement, sequences, lead score, Conversations. Use a private app token or OAuth — your call. Seed it with realistic data.
A database of your choice (Supabase fits; spec assumes it)	Your isolated program stores in Phase 1; app_form-style funnel data and the Hub's own state (ideas, budgets, widget layouts) in Phase 2.
Stripe (test mode)	Payments in Phase 1. Use webhooks to drive state changes through the system.
Open Data (tryopendata.ai)	External public-school data — Texas PEIMS finances, STAAR, accountability ratings. Free tier, API + SDK + MCP, no signup needed.
A hosting / runtime of your choice	The spec suggests Next.js on Vercel; you may choose otherwise and justify it. Must be runnable by us.

An AI layer is encouraged — e.g. the spec's brand-voice auditor (suggest-edits mode), SMS auto-theme tagging, or an Ask-the-Hub agent over the combined data. If you add services beyond these, have a reason.

Your test data is part of the deliverable

We do not ship you a dataset. Designing, generating, and validating realistic data — for the sources you can reach (HubSpot, Stripe, your DB) and the ones you stand in for (Meta, GA4, X, summer.gt.school, community.gt.school, Google Sheets) — is itself something we score. We want to see how you think about data, not just how you wire it.

- Model it like the real thing: families, deals, enrollments, engagement, budgets and segments that follow the spec's shapes, owners, and the $365K budget — not lorem-ipsum rows.
- Volume and spread that make the work non-trivial: enough records, across enough states and time, that your reconciliation, dashboards, and pacing math are actually exercised. A handful of happy-path rows is a red flag.
- Edge cases on purpose: duplicates, a family in two programs, a late/failed/duplicate payment, conflicting CRM vs. app values, a sync-parity drop, mojibake or missing fields. Your isolation, idempotency, and dual-source reconciliation should be provable against data you built to stress them.
- Make it reproducible and honest: ship your seed/generation scripts (or fixtures), label clearly what is real vs. stood-in, and be able to reset to a clean known state for the walkthrough.

Where to spend your depth: the module menu

Two days is nowhere near enough for all 13 modules, and trying is a red flag. The judgment we are scoring is which modules you build deep, which you stub or skip, and why. Below is the spec's module set with what each one really tests, so you can choose deliberately. Build a small number exceptionally well, wired into the shared backbone and honoring the cross-module rules — then defend what you cut in your write-up.

Home / Executive Command Center	Composable per-user dashboard from a 30+ widget library, with a default starter pack and saved layouts. Tests aggregation across modules and per-user state. A strong spine for everything else.
Budget Tracker (Budget Owner)	Workstream rows reconciling to the $365K total; burn chart; >10% variance auto-flags to the Decision Queue. The clearest test of 'a number means the same thing everywhere.'
Decision Queue (Leaders only)	Async approve / reject / need-info workflow, gated to leadership, fed by submissions and auto-flags from other modules. The cleanest role-gating + cross-module test.
Nurture & Lifecycle (Marketing Lead)	The most data-rich module: T1/T2/T3 segments, engagement-tier × attribute heatmap, HubSpot pipeline stages, SMS inbox with auto-theme tagging, 24-hr SLA. Deep HubSpot + Supabase integration.
CRM / Marketing Operations (Marketing Lead)	Sync parity score, UTM health (known broken), data-quality queue with auto-detection, field-reliability flags. This is where your Phase 1 backbone surfaces as product — a natural pairing.

Dashboard / KPI Tracking (Marketing Lead)	The canonical weekly scorecard (shared, identical for all), goal pacing projections, HubSpot dashboard mirror. Reads from everything, owns nothing — tests your single-source-of-truth discipline.
Grassroots Engine (Grassroots Owner)	Ambassador pipeline, market map, referral sprints, parent-led event calendar. Dual-source reconciliation (HubSpot + community.gt.school) and rich cross-links to Content and Admissions.
Content & Thought Leadership (Content Owner)	Production kanban synced to a Google Sheet (read+write), content calendar, per-piece UTM attribution, brand-voice auditor in suggest-edits mode. Good place for an AI layer.
Admissions & Voice of Customer (Admissions Owner)	Objection log with trends, objection-to-content bridge (auto-stub briefs into Content), family sentiment. Tests the auto-created cross-link rules.
Summer Camp (Content Owner)	Per-campus capacity, registration funnel, revenue vs. target — reconciling summer.gt.school + a registration form without double-counting. The clearest dual-source test, and it ties to Phase 1's program isolation.
Field Marketing & Events (Field & Events Owner)	Event tracker + calendar, with a read-only cross-link to ambassador-hosted events that LIVE in Grassroots. Tests cross-module ownership boundaries.
Website & Digital Analytics	GA4 across two sites, subpage performance, PDF download tracking, conversion paths. Mostly an integration + viz module; easy to stub with seeded data.
Resource Library	A flat, tag-filterable document shelf. Deliberately simple — a reasonable place to stub and move on.

In your write-up, show the modules you considered and your reasoning for what made the cut. 'I built CRM Ops + Budget + Decision Queue end-to-end on a real backbone, and here's why those three' is a strong answer. 'I touched all 13 shallowly' is not.

A worked example: the GT Challenge

To show the whole loop end-to-end — capture, assess, reconcile, report — here is one real GT play you can build into the Hub. It is an illustration of an end-to-end campaign the system should support, not a required module.

> The GT Challenge is a free, short assessment — a gifted-style quiz — that GT publishes on social media as a lead magnet. A parent sees the post, their child takes the Challenge, and in exchange GT captures a lead and returns a result. It is simultaneously a marketing campaign (with spend, a channel, and a CAC) AND a product surface that generates submissions GT's own program then assesses — scoring each submission, bucketing the child, and routing strong fits into the admissions funnel.

What building it into the system looks like

- Stand it up as a campaign with budget: a real entry whose spend rolls into the Budget Tracker's workstream totals — same reconciliation rules as every other line.
- Capture submissions: a lightweight public quiz whose responses flow into your data store and become leads in HubSpot — routed into the right program store via the Phase 1 backbone, with UTM captured (so it shows up honestly in CRM Ops).
- Let the program assess it: auto-score each submission (rules or an AI grader), bucket the result, write the outcome back — no submission lost, no double-counting, no cross-contamination.
- Close the loop: a Home widget / KPI row showing the Challenge as a campaign — spend, submissions, qualified leads, cost-per-qualified-lead — beside other channels, optionally enriched with Open Data.

Done well, the GT Challenge exercises every theme at once: the sync backbone moving real lead and budget data without contamination, an AI/program assessment step, the single-source-of-truth and budget rules, and one view where the numbers reconcile. An excellent — but optional — way to show the whole system working end to end.

Show us it works

Across both phases, give us a way to see it working — the Hub UI itself, an admin view, or a clear API. Polish matters less than honesty: we should be able to watch a payment propagate, a budget reconcile to the total, a role be denied the Decision Queue, and the data-confidence banner appear when parity drops.

Where you can go further (optional, unscored-but-noticed)

Strong directions, all rooted in the spec: a conflict-resolution strategy with an audit trail; row-level security or per-tenant key isolation you can demonstrate defeating an attempted leak; a replayable event log; automated tests that simulate webhook storms and race conditions; the brand-voice auditor; LLM-based SMS auto-theming (the spec's v2); an Ask-the-Hub agent answering 'which channel should we double down on, given our CAC and the local market?' across HubSpot and Open Data. Pick what matters most and own that call.

Ground rules

- The attached spec is the requirement for Phase 2. Read it fully first. Where it specifies (modules, roles, data sources, source-of-truth rules), implement it. Where it leaves an engineering choice open, decide and defend.

- Real services, your own accounts. Never commit secrets to git. How you manage credentials is evaluated.
- Build Phase 1 first — Phase 2 should consume it, not re-implement it. The Hub must stand on the backbone.
- You will not finish all 13 modules. Sequencing is part of the test: go deep on a few, stub the rest deliberately, and document why.
- Use any language, framework, and AI tooling you want. Tell us what you used, and which decisions your tools made vs. you.
- Where you make an assumption or fill a gap the spec leaves open, write it down — we want to see your judgment.

What to submit

Code	A git repository (link or zip). Include a README that gets us running in minutes.
Write-up (1–2 pages max)	What you built, which spec modules you went deep on vs. stubbed and WHY, key technical trade-offs, where you honored (or had to bend) the spec's rules, and what you'd do with another week.
Proof it works	How you tested it — especially data isolation + idempotency + dual-source reconciliation in Phase 1, and budget reconciliation, role gating, and live integrations in Phase 2. Tests, scripts, or a clear manual procedure.
Walkthrough video (5–10 min)	A required screen recording (with your voice narrating) driving the system end-to-end: a payment propagating without contamination; logging in as a Leader vs. an Operator and showing the Decision Queue gate; a budget edit reconciling to the total; and a real Open Data query — including at least one failure/edge case. Upload to YouTube (unlisted), Loom, or a shared drive and include the link.

How to submit

> Send everything to Logan.may@superbuilders.school by Saturday night (11:59 PM Central). In one email include: (1) the git repository link (or a zip attachment); (2) your write-up (PDF or a link); (3) the walkthrough video link; and (4) any live demo URL plus the credentials for the three test roles (Admin, Leader, Operator). Use the subject line "GT Technical Project — [Your Name]." If anything is still deploying at the deadline, send what you have on time and note what is in flight — we would rather see an honest, on-time submission than a late one.

How we'll read it

> We are not counting modules. We are looking for an engineer who can take a real, detailed, messy specification and a multi-service backbone, and execute the right slice of it with judgment — honoring the source-of-truth rules, reconciling dual sources, gating by role, and being honest about what is broken — then prove the numbers are right. Show us how you read a spec, how you sequence it, how you own it, and how you test it.